

NumberBot

Last updated: 2026-05-11

NumberBot is ArtHound's production-data AI assistant, powered by Claude Haiku. It answers questions about a studio's or vendor's asset inventory, work estimates, and review state, grounded strictly in live ArtHound data. It refuses questions outside its defined scope.

What It Is

NumberBot is a conversational assistant surfaced as a chat panel in the ArtHound UI. It is not a general-purpose assistant. Its purpose is narrow: help studios and vendors interrogate their own production data without writing queries or building reports manually.

Example questions it is designed to answer: - "How many character assets do we have by product?" - "What's the total estimated days of rigging work across open assets?" - "Which assets have a review status of 'Changes Requested'?" - "Show me all assets with no estimate in the matrix."

Data Access

Before each conversation turn, NumberBot fetches live context in parallel from ArtHound's database:

Context	Source
Asset inventory	<code>replicated_assets</code> , name, item type, product, status, priority, raw fields
Field mappings	<code>source_field_mappings</code> , slot assignments and field labels
Source work	<code>replicated_work</code> , synced work items with status and estimates
Generated work	<code>generated_work</code> , ArtHound-generated snapshots (deleted_at IS NULL)
Asset reviews	<code>asset_reviews</code> , review titles and statuses for the org

From the asset and field data, NumberBot builds an ASCII context table showing: - Asset breakdown by product, item type, and status - Field mapping reference (slot → source field name) - Work list with estimate values - Filtered meta keywords (milestone, date, team, phase, due, target, delivery, these are included when present to help with scheduling questions)

This context is passed to the model as a system prompt with [prompt caching](#) enabled (`cache_control: {type: "ephemeral"}`), reducing latency and cost on follow-up turns within the same session.

Scope Rules

NumberBot operates under strict scope rules enforced in its system prompt:

1. Only answer questions about the production data in the provided context
2. Do not speculate about data not present in the context
3. Do not answer general knowledge, business advice, or coding questions
4. If asked something outside scope, state clearly that it can only discuss production data and decline to answer

If a user asks "What's the best rigging approach for a character with 80,000 polys?", NumberBot declines and redirects to production data queries.

Model and API

- **Model:** Claude Haiku 4.5 (`claude-haiku-4-5-20251001`)
- **Route:** `POST /api/numbersbot/chat` (note: route file is `routes/numbersbot.py`)
- **Auth:** JWT required; scoped to the calling user's studio or vendor

Request body:

```
{
  "messages": [
    {"role": "user", "content": "How many assets are in the Character category?"},
    {"role": "assistant", "content": "..."},
    ...
  ]
}
```

The route prepends the live context as a system message before passing the conversation to the Anthropic API. The model's response is streamed back to the client.

Prompt Caching

The context block (asset inventory, field mappings, work and review data) is marked with `cache_control: {type: "ephemeral"}`. Anthropic's prompt cache has a 5-minute TTL. Subsequent turns in the same session within that window reuse the cached context, making follow-up questions significantly faster and cheaper.

The cache is keyed on the exact content of the context block. If the underlying data changes (e.g., a sync runs between turns), the context will differ and the cache will miss, this is correct behaviour, as the user should see fresh data.

Known Gaps

source_entity_definitions not injected, NumberBot's context does not include the studio's P→A→W hierarchy definition. Questions about which source table maps to which ArtHound entity cannot be answered accurately.

No pagination on context fetch: the context fetch loads all assets and work items for the org. For large studios (thousands of assets), this context block may grow very large, increasing latency and potentially hitting model context limits.

Session context is per-request: NumberBot has no persistent conversation memory beyond what the client sends in the `messages` array. The client is responsible for maintaining conversation history across turns.

Vendor access is limited: vendors only see assets they have received via dispatches. The context fetch is scoped accordingly, but this means vendor NumberBot sessions have materially less data to work with than studio sessions.